CSC-239
Java Programming:
Chapter 2: Elementary Programming

September 8, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder

- Quiz 1 will begin on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will open Monday evening, at 9:00 p.m.
  - The Quiz will remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Reading Input from the Console

1. Create a Scanner object

```
Scanner input = new Scanner(System.in);
```

2. Use one of the class methods to obtain a value of the desired type. For example:

```
System.out.print("Enter a double value: ");
double d = input.nextDouble();

System.out.print("Enter an integer value: ");
double i = input.nextInt();
```

3. If the user enters text which does not comply with the requested type, a runtime exception will result. (We will talk more about this in an upcoming lecture.)

Declaring Variables

- Declaring primitive variables is similar to other languages:

```
int number;     // integer variable
double radius; // double-precision
                // floating point variable;
char letter;    // single character
```

Assignment Statements

```
number = 1;      // Assign 1 to number;

radius = 1.0;    // Assign 1.0 to radius;

letter = 'A';    // Assign 'A' to letter;
```

It is possible to declare and initialize a variable in one step:

```
int x = 1;

double d = 1.4;
```

Identifiers (Names)

- An identifier is a sequence of characters that consist of letters, digits, underscores (_), and dollar signs ($).
- An identifier must start with a letter, an underscore (_), or a dollar sign ($). It cannot start with a digit.
  - An identifier cannot be a reserved word. (See Appendix A, "Java Keywords," for a list of reserved words).
  - Some groups follow a convention that identifiers starting with an underscore (_) are used only in software-generated code. For this reason, I discourage using names that begin with an underscore.
- An identifier cannot be `true`, `false`, or `null`.
- An identifier can be of any length.

Named Constants

- To declare a constant, add the keyword "`final`" to the definition:

`final` *datatype CONSTANT_NAME = VALUE*`;`

- For example:

```
final double PI = 3.14159;
final int SIZE = 3;
```

Naming Conventions

- Choose meaningful and descriptive names.
- Variables and method names:

  Use lowercase. If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name. For example, the variables `radius` and `totalArea`, and the method `computeArea`.

Naming Conventions, cont.

- Class names:

  Capitalize the first letter of each word in the name. For example, we might have a class named **`GeometricShape`**, which creates objects that represent two-dimensional geometric objects.

- Constants:

  Capitalize all letters in constants, and use underscores to connect words. For example, the constants **`PI`** and **`MAX_VALUE`**

Numerical Data Types

Name	Range	Storage Size
`byte`	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
`short`	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
`int`	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
`long`	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
`float`	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
`double`	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2

Integer Division

`5 / 2` evaluates to an integer value of `2`.

`5.0 / 2` evaluates to a double value of `2.5`

`5 % 2` evaluates to `1`

(the remainder of the integer division)

- The "remainder" operation is also called "modulo" or simply "mod":

  "5 mod 2 evaluates to 1"

Remainder (Modulo) Operator

- Remainder is very useful in programming.
- For example, an even number % 2 is always 0 and an odd number % 2 is always 1. So you can use this property to determine whether a number is even or odd.
- The Remainder operator (%) is sometimes called "modulus", "modulo" or "mod". This is in reference to the "modulus" concept in mathematics. Like many concepts in computer science, there are sometimes multiple names for the same thing!

Remainder Exercise

- For the sake of discussion, let us number the days of the week:

  Sun = 0, Mon = 1, Tue = 2, Wed = 3,

  Thu = 4, Fri = 5, Sat = 6

- Suppose today is Saturday and you and your friends are planning to meet in 10 days. What day of the week is in 10 days?
- Can you write Java statements to calculate this?

Remainder Exercise -- Solution

- For the sake of discussion, let us number the days of the week:

  Sun = 0, Mon = 1, Tue = 2, Wed = 3,

  Thu = 4, Fri = 5, Sat = 6

- Suppose today is Saturday and you and your friends are planning to meet in 10 days. What day is in 10 days?
- Can you write Java statements to calculate this?

```
today = 6; // today is Saturday
tenDays = (today+10)%7; // Tuesday
```

Numerical Precision

- Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy. For example,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

displays `0.5000000000000001`, not `0.5`, and

```
System.out.println(1.0 - 0.9);
```

displays `0.09999999999999998`, not `0.1`.

- Integers are stored precisely. Therefore, calculations with integers yield a precise integer result.

Number Literals

A *literal* is a constant value that appears directly in the program.

For example: **`34`**, **`1000000`**, and **`5.0`** are literals in the following statements:

```
int i = 34;
long x = 1000000;
double d = 5.0;
```

Integer Literals

- An integer literal can be assigned to an integer variable as long as it can fit into the variable. A compilation error would occur if the literal were too large for the variable to hold.
- For example, the statement

```
byte b = 1000;   // (compiler error)
```

  would cause a compilation error, because 1000 cannot be stored in a variable of the byte type.

- By default, an integer literal is assumed to be of the int type.
- To denote an integer literal of the long type, append it with the letter **L** or **l**. Uppercase **L** is preferred because **l** (lowercase **L**) can easily be confused with **1** (the digit one).

Floating-Point Literals

- Floating-point literals are written with a decimal point.
- By default, a floating-point literal is treated as a double type value.
- You can make a floating point literal a float by appending the letter f or F.

  For example: `100.2f` or `100.2F`

- You can explicitly indicate that a number is a double by appending the letter d or D.

  For example: `100.2d` or `100.2D`

Scientific Notation (1)

- Floating-point literals can also be specified in scientific notation.
- For example, these three literals represent the same value:

```
1.23456e+2        1.23456e2        123.456
```

- Also `1.23456e-2` is equivalent to `0.0123456`.
- E (or e) represents an exponent and it can be either in lowercase or uppercase.

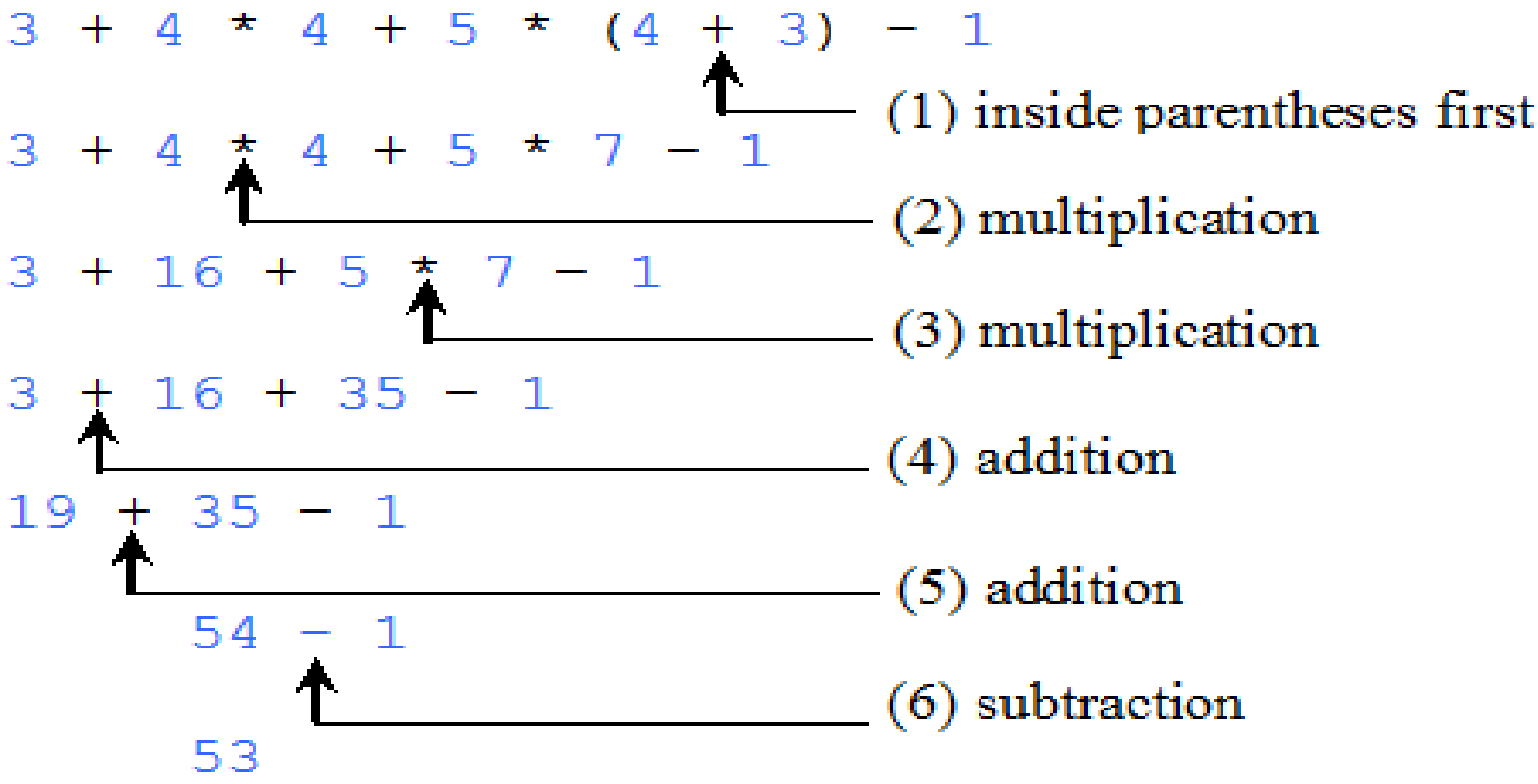
Scientific Notation (2)

- Sometimes people need to work with numbers that are either very large or very small.
- Rather than write numbers with many digits, a standard called *Scientific Notation* is used.
- Some examples:

Number	Scientific Notation	Expressed in Java
10	1.0×10^{1}	1.0E+1
100	1.0×10^{2}	1.0E+2
0.15	1.5×10^{-1}	1.5E-1
3500000	3.5×10^{6}	3.5E+6
32000	3.2×10^{4}	3.2E+4
0.000053	5.3×10^{-5}	5.3E-5

How to Evaluate an Expression

- When Evaluating an expression, follow the Operator Precedence Chart in Appendix C.



Shortcut Assignment Operators

Operator	Example	Equivalent
`+=`	`i += 8`	`i = i + 8`
`-=`	`f -= 8.0`	`f = f - 8.0`
`*=`	`i *= 8`	`i = i * 8`
`/=`	`i /= 8`	`i = i / 8`
`%=`	`i %= 8`	`i = i % 8`

Increment and Decrement Operators

Operator	Name	Description
`++var`	preincrement	The expression (**`++var`**) increments **var** by 1 and evaluates to the *new* value in **var** *after* the increment.
`var++`	postincrement	The expression (**`var++`**) evaluates to the *original* value in **var** and then increments **var** by 1.
`--var`	predecrement	The expression (**`--var`**) decrements **var** by 1 and evaluates to the *new* value in **var** *after* the decrement.
`var--`	postdecrement	The expression (**`var--`**) evaluates to the *original* value in **var** and then decrements **var** by 1.

Increment / Decrement Operators

- Using increment and decrement operators makes expressions shorter, but it also sometimes makes them complex and difficult to read.
- Avoid using these operators in expressions that modify multiple variables, or the same variable multiple times such as this:

```
int k = ++i + i;
```

  Instead, write a sequence of two or more steps that accomplish the same thing, but avoid confusing the reader.

- Remember: if it is tricky to read, then it will also be tricky to be sure it works correctly.

Numeric Type Conversion

- Consider the following statements:

```
byte i = 100;
long k = i * 3 + 4;
double d = i * 3.1 + k / 2;
```

- How does Java handle expressions involving values of different types?

Type Conversion Rules

When performing an operation involving two operands of different numeric types, Java automatically converts the operand based on the following rules:

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.

Type Casting

- Implicit casting:

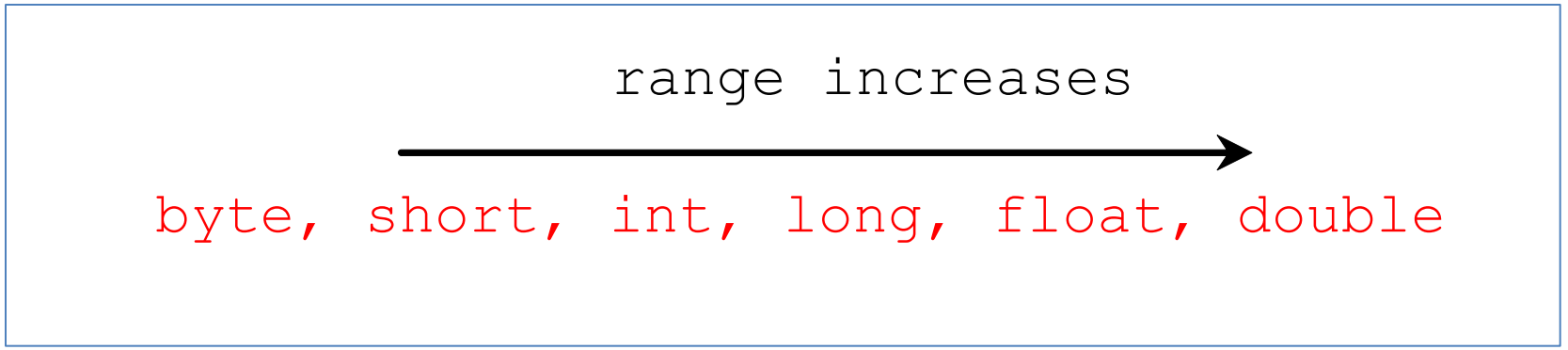
```
double d = 3;  //(type widening)
```

- Explicit casting:

```
int i = (int)3.0; // (type narrowing)
int i = (int)3.9; // (Fractional part
                  //   is truncated)
```

- What is wrong with this?

```
int x = 5 / 2.0;
```



Character Data Type

- In Java, each character is internally represented by 16 bits (not 8 bits)
  - Unicode character set
- Historically, most software tools and operating systems have used characters that have 8 bits.
  - ANSI or ASCII character set.
- This difference is not usually a big problem, because the compiler and standard input/output libraries do the conversion for us.

Character Literals

- The Unicode equivalent of the ASCII characters start with 8 zero bits

```
char letter='A';          // (ASCII)
char letter='\u0041';     // (Unicode for ASCII 'A')
char numChar='4';         // (ASCII)
char numChar='\u0034';    // (Unicode for  ASCII '4')
```

- The extra bits are used to express character sets for non-english languages, and for graphical characters.

Escape Sequences for Special Characters

Description	Escape Sequence	Unicode
Backspace	`\b`	`\u0008`
Tab	`\t`	`\u0009`
Linefeed	`\n`	`\u000A`
Carriage return	`\r`	`\u000D`
Backslash	`\\`	`\u005C`
Single Quote	`\'`	`\u0027`
Double Quote	`\"`	`\u0022`

Casting between `char` and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';

char c = 97; // Same as char c = (char)97;
```

The String Type

- The **char** type only represents one character. To represent a string of characters, use the data type called **String**. For example,

```
String message = "Welcome to Java";
```

- String is actually a *predefined* class in the Java library, in the same way that the System class is a predefined class.
- The String type is not a primitive type. It is known as a *reference type*. Any Java class can be used as a reference type for a variable. Reference data types will be thoroughly discussed in Chapter 9, "Objects and Classes."
- For the time being, you just need to know how to declare a String variable, how to assign a string to the variable, and how to concatenate strings.

  (Refer to the next several slides.)

String Concatenation

- Three strings are concatenated:

```
String message = "Welcome " + "to " + "Java";
```

- String "Chapter" is concatenated with number 2:

```
String s = "Chapter" + 2; // s becomes "Chapter2"
```

- String "Supplement" is concatenated with character B:

```
String s1 = "Supplement" + 'B'; // s1 becomes
                                // "SupplementB"
```

Converting Strings to Integers

- To convert a string into an int value, you can use the static parseInt method in the Integer class as follows:

```
int intValue = Integer.parseInt(intString);
```

where intString is a numeric string such as “123”.

Converting Strings to Doubles

- To convert a string into a double value, you can use the static parseDouble method in the Double class as follows:

```
double doubleValue =Double.parseDouble(doubleString);
```

where doubleString is a numeric string such as “123.45”.

Lab01a: *GraderThan* environment

- Follow the instructions in the **Lab01a** document. This will help you get familiar with the *GraderThan* tool set.
- You may need to use the *GraderThan* environment to complete some future assignments.